



Lab Experiment 04: Real-Time Operating System

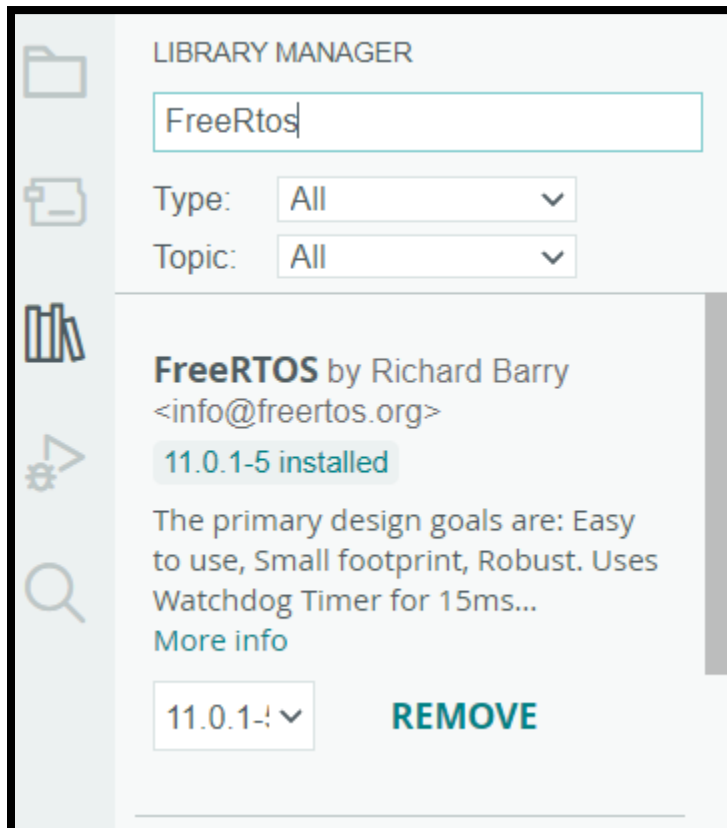
Objectives

- Getting introduced to real-time operating system on Arduino (FreeRTOS usecase).

Hint: You can find the lab repository: <https://drive.google.com/drive/folders/1zF3k1eYjMVVLnccmwKSIHo-iTeV5hWsa?usp=sharing>

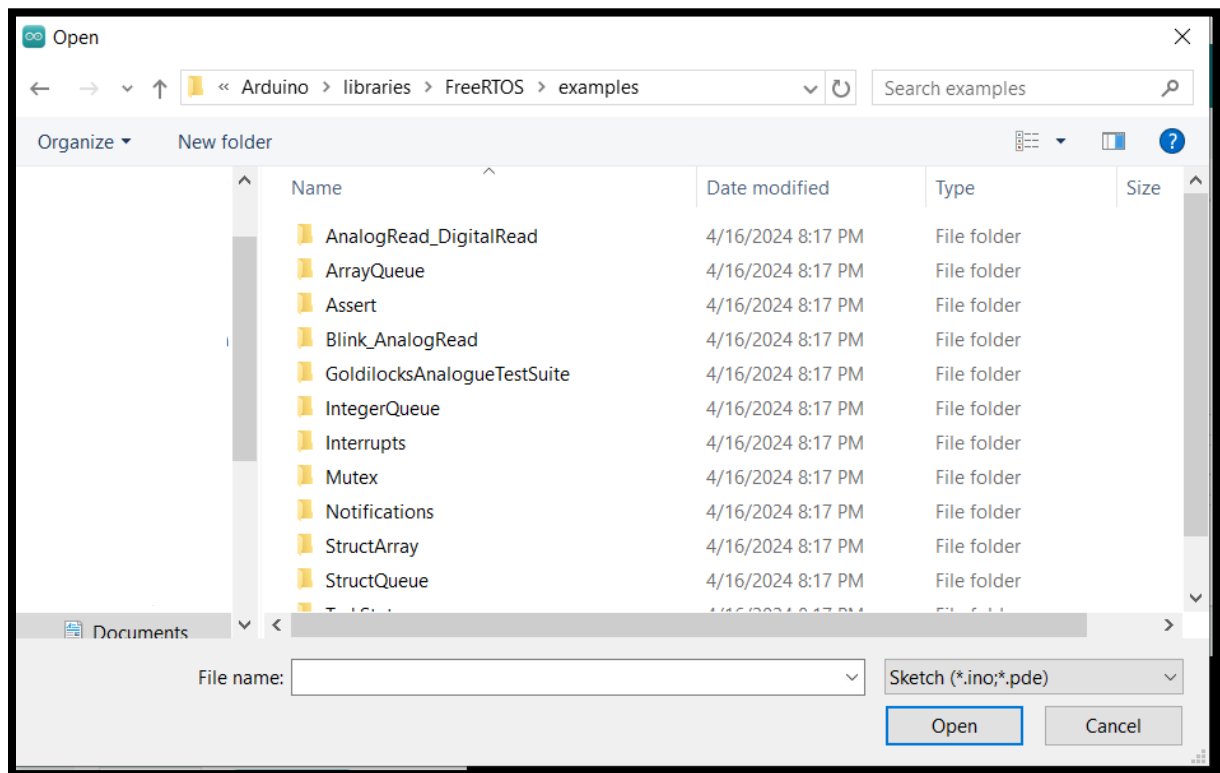
Procedure

- Install latest version of FreeRTOS library in the Arduino IDE.



- Try one of the examples we discussed in the lab sessions. They are listed under the *Examples* folder in library.

- AnalogRead_DigitalRead
- ArrayQueue
- Blink_AnalogRead
- TaskStatus
- TaskUtilities



- The corresponding run samples are found under the *Examples/runs* folder in repository.
- List the files you used for compilation. You can find them under:
C:\Users\<Username>\AppData\Local\Temp\arduino\sketches\<id>\libraries\FreeRTOS
Hint: Use the most recent date.
Eg:
C:\Users\salah\AppData\Local\Temp\arduino\sketches\CE41AC306EC2C925E1BD572EDBCAA328\libraries\FreeRTOS
- List the storage footprint. You can find it at the bottom of the Arduino IDE compile output.

Question

Answer the following exercise and attach your solution to the report (The solution is graded as part of the assignment)

The default RTOS scheduling policy (single-core)

FreeRTOS uses a fixed-priority preemptive scheduling policy, with round-robin time-slicing of equal priority tasks:

"Fixed priority" means the scheduler will not permanently change the priority of a task.

"Preemptive" means the scheduler always runs the highest priority RTOS task that is able to run, regardless of when a task becomes able to run. For example, if an interrupt service routine (ISR) changes the highest priority task that is able to run, the scheduler will stop the currently running lower priority task and start the higher priority task - even if that occurs within a time slice. In this case, the lower priority task is said to have been "preempted" by the higher priority task.

"Round-robin" means tasks that share a priority take turns entering the Running state.

"Time sliced" means the scheduler will switch between tasks of equal priority on each tick interrupt - the time between tick interrupts being one time slice. (The tick interrupt is the periodic interrupt used by the RTOS to measure time.)

A system has 3 tasks A, B, C. with priorities 1, 2, 1. the need 2, 5, 3 ticks to execute. the ready list at start is A, B, C.

tasks A, C never block. task B executes one tick and blocks 2ticks.

give a table showing at each tick which task is executing using the above sechduling policy.

N.B.: all tasks block for ever after finishing.

References

- Introduction to FreeRTOS
Link: <https://en.wikipedia.org/wiki/FreeRTOS>
- FreeRTOS Official Website
Link: <https://www.freertos.org/index.html>
- Getting started with FreeRTOS API
Link: https://docs.google.com/document/d/1_jqUBYkocESLIzQEpfUwR6h1XUR0n-du/edit?usp=sharing&oid=102450282224179968221&rtpof=true&sd=true
- FreeRTOS Architecture
Link: https://drive.google.com/file/d/1Ml_mqLzgwkyS6Dn4dgMqlzYXeRZv6s34/view?usp=sharing
- FreeRTOS github repository
Link: <https://github.com/FreeRTOS/FreeRTOS.git>

Delivery Policy

- You should work in groups of two members.
- Each group must send a report showing the run output in a screenshot form of the program they chose to try.
- Each group must submit the solution to the theoretical part in the report.
- Each group must submit the list of files compiled and the storage footprint in the report.
- You should submit a report showing the challenges you faced (if any) and the comments you have on results.
- You should cite any additional resources you used.

- Further details for the submission instructions will be posted later on MS Teams.

Good Luck